

Universidade da Beira Interior

Departamento de Informática



Departamento de  
Informática

Nº 1 - 2026: *[Relatório Técnico: Jogo da  
Força Multijogador]*

Elaborado por:

[Clausemen Nanro Nº 53269 ]  
[João Loureiro Nº 52645]

Orientador:

Tiago Simões

April 19, 2026

19 de abril de 2026

# Conteúdo

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introdução</b>                                                       | <b>3</b>  |
| 1.1      | Contextualização . . . . .                                              | 3         |
| 1.2      | Objetivos do Trabalho . . . . .                                         | 3         |
| <b>2</b> | <b>Arquitetura e Design</b>                                             | <b>4</b>  |
| 2.1      | Modelo Cliente-Servidor . . . . .                                       | 4         |
| 2.2      | Protocolo de Comunicação . . . . .                                      | 4         |
| 2.3      | Fluxo de Jogo e Sincronização . . . . .                                 | 4         |
| 2.3.1    | Estados do Jogo . . . . .                                               | 4         |
| <b>3</b> | <b>Análise da Implementação</b>                                         | <b>6</b>  |
| 3.1      | Estrutura do Servidor e Gestão de Threads . . . . .                     | 6         |
| 3.1.1    | Abertura do servidor para clientes e a sua gestão via threads . . . . . | 6         |
| 3.1.2    | Ciclo de vida da thread . . . . .                                       | 7         |
| 3.2      | Controlo de Concorrência e Secções Críticas . . . . .                   | 7         |
| 3.2.1    | Mecanismo de sincronização . . . . .                                    | 7         |
| 3.3      | Implementação de Timeouts . . . . .                                     | 8         |
| 3.3.1    | Constantes de timeouts . . . . .                                        | 8         |
| 3.4      | Interface e Lógica do Cliente . . . . .                                 | 8         |
| 3.4.1    | Lógica de execução e leitura de mensagens . . . . .                     | 8         |
| 3.4.2    | Inicialização da interface gráfica . . . . .                            | 9         |
| <b>4</b> | <b>Demonstração do Jogo</b>                                             | <b>11</b> |
| 4.0.1    | Início do Jogo . . . . .                                                | 11        |
| 4.0.2    | Aguardar Jogadores . . . . .                                            | 11        |
| 4.0.3    | Jogo em andamento . . . . .                                             | 12        |
| 4.0.4    | Cenário de vitória . . . . .                                            | 13        |
| 4.0.5    | Cenário de derrota . . . . .                                            | 13        |

|          |                                    |           |
|----------|------------------------------------|-----------|
| <b>5</b> | <b>Conclusão</b>                   | <b>15</b> |
| 5.1      | Dificuldades Encontradas . . . . . | 15        |
| 5.2      | Considerações Finais . . . . .     | 15        |

# Capítulo 1

## Introdução

Este relatório foi feito no âmbito do trabalho prático 1 de sistemas distribuídos, no qual foi desenvolvido um jogo da forca, implementando sockets TCP/IP, sincronização de rondas, á prova de race conditions, validações de entrada, entre outras coisas.

### 1.1 Contextualização

No âmbito da componente letiva da disciplina, foi proposta a criação de uma arquitetura cliente-servidor que integrasse os conceitos fundamentais explorados nas aulas. O projeto foca-se na gestão dinâmica de jogadores e na implementação de protocolos de comunicação robustos para o controlo de estados de jogo, como vitória, empate ou derrota.

### 1.2 Objetivos do Trabalho

O objetivo deste trabalho foi aplicar os conhecimentos aprendidos nas aulas teóricas e práticas de Sistemas distribuídos, para assim, ter uma maior consolidação das matérias a nível mais profundo e aprender como podemos usar isto no nosso trabalho futuro.

# Capítulo 2

## Arquitetura e Design

### 2.1 Modelo Cliente-Servidor

Nós aqui optamos por usar o modelo Cliente-Servidor, que é o modelo mais usado na internet hoje em dia, devido a sua facilidade de implementação e simplicidade de conectar vários jogadores numa mesma partida. Pela sua rapidez, ela é amplamente usada em várias app's em que rapidez é fundamental, resultando no uso global que vemos nos dias de hoje.

### 2.2 Protocolo de Comunicação

Dos protocolos de comunicação optamos por usar TCP/IP, por ser o protocolo de comunicação mais usado do mundo e por na nossa opinião, se tratar do melhor protocolo para o jogo da força. Isso se deve á sua ordenação de pacotes, garantia de entrega, facilidade de integração e manter a comunicação aberta que é necessário neste jogo, onde tê-la sempre aberta significa maior rapidez, porque não precisa de abrir uma nova ligação.

### 2.3 Fluxo de Jogo e Sincronização

Para maior facilidade de gestão e manutenção, nós optamos por usar um fluxo de jogo conciso e organizado. Com isso, conseguimos uma maior escalabilidade, desempenho e segurança, garantindo que cumpríamos todos os pilares principais da programação.

#### 2.3.1 Estados do Jogo

Nos estados do jogo, acabamos por dividir em 3 estados:

A aguardar jogadores → Em progresso → Terminado

Com isso, conseguimos uma maior organização no código.

### **1 - Entrada e junta de jogadores na mesma partida**

Nesta primeira fase é quando os jogadores entram e esperamos que passe X tempo ou que o lobby fique completo. Se o jogo já estiver cheio, o jogador recebe a mensagem "FULL". utiliza "synchronized"+ "notifyAll()" para avisar o lobby de novas entradas.

### **2 - Inicio do jogo**

Temos um thread dedicada apenas ao inicio do jogo "runLobby" e o que ela faz é especificamente controlar quando o jogo começa, que neste caso é quando o jogo tem 2 jogadores ou mais. Após entrar o primeiro jogador, a thread faz o jogador esperar 20 segundos que entrem mais(até 4), se isso não acontecer, ou seja, se ninguém entrar para além do primeiro jogador, o jogo fica "pausado" até que alguém entre.

### **3 - Rondas síncronas**

1. Antes de cada ronda, o servidor obtém os jogadores ativos no momento.
2. Cada jogador é informado do inicio da ronda e o estado atual.
3. Cada jogador efetua a sua jogada ou deixa vazio
4. Após o tempo acabar, são calculadas as pontuações e o servidor envia o resumo da ronda
5. Depois temos um loop que roda todas as rondas

# Capítulo 3

## Análise da Implementação

### 3.1 Estrutura do Servidor e Gestão de Threads

Como todas as grandes aplicações multi-utilizador fazem, nós optamos por também usar threads para fazer a gestão do jogo.

#### 3.1.1 Abertura do servidor para clientes e a sua gestão via threads

Neste trecho de código, temos um *while(true)* que logo após processar um cliente, já fica á espera do próximo. O *Socket socketCliente = servidorSocket.accept();* espera que o próximo cliente entre, enquanto isso não acontecer, a função fica bloqueada nessa linha até que alguém apareça. Após isso, criamos uma nova gestão de cliente e depois o servidor entrega a tarefa às threads. O uso de threads tem como objetivo fazer o servidor conseguir suportar múltiplas conexões de clientes, evitando assim o cliente ter de ficar á espera da sua vez.

```
while (true) {
    Socket socketCliente = servidorSocket.accept();
    System.out.println("[Servidor] Nova ligação de"+
        socketCliente.getInetAddress().getHostAddress());
    ;

    GestorCliente gestor =
    new GestorCliente(socketCliente, gameManager);
    conjuntoThreads.submit(gestor);
}
```



### 3.1.2 Ciclo de vida da thread

Aqui o servidor fica a ouvir o cliente até que a thread seja encerrada. Ele controla todos os input's digitados pelo utilizador, de modo a tornar o jogo fluido e jogável de maneira suave e rápida. Mal a thread do cliente seja fechada, o *ativo = false*, logo o jogo para de captar aquele cliente em específico.

```
while (ativo) {
    try {
        String line = readLine();
        if (line == null) {
            break;
        }
        processMessage(line.trim());
    } catch (SocketTimeoutException e) {
        // Sem mensagem neste intervalo: continuar a
        // aguardar.
        // A ronda resolvida no GestorJogo com
        // CountdownLatch +
        // timeout.
    }
}
```

...

## 3.2 Controlo de Concorrência e Secções Críticas

### 3.2.1 Mecanismo de sincronização

Na primeira parte do código, temos uma parada, em que quando o jogo começa já não podem entrar mais jogadores. Isso evita incoerências, e evita a concorrência e condições de corrida, porque ele lê os jogadores quando já está fechado, e assim garante a leitura de todos os jogadores de maneira correta.

No segundo código, temos uma das partes mais críticas do código e a função dela é garantir que os resultados só saem quando todos os jogadores tiverem jogado ou quando o tempo da ronda acaba, garantindo assim, um jogo justo que é a base de quase todos os jogos atuais.

```
synchronized (this) {
```

```

        joiningOpen = false;
        instantaneo = new ArrayList<>(handlers);
    }

    ...
    roundLatch = new CountDownLatch(activeIds.size());
    try{
        boolean complete = roundLatch.await(EstadoJogo.
            ROUND_TIMEOUT_MS, TimeUnit.MILLISECONDS);
    }

```

## 3.3 Implementação de Timeouts

### 3.3.1 Constantes de timeouts

No código, usamos vários timeouts de modo a definir tempos limites para ações, ou números máximos de algo, por exemplo, jogadores, entre outras coisas. Isto, tem como objetivo deixar o código mais limpo e aumentar a facilidade em alterar as regras do jogo.

```

// De EstadoJogo.java (linhas ~20-30)
public static final int MAX_PLAYERS = 4;
public static final int MAX_ATTEMPTS = 6;

/** Tempo (em ms) que cada jogador tem para submeter a
    sua jogada por ronda. */
public static final int ROUND_TIMEOUT_MS = 15_000;

/** Tempo (em ms) de espera no lobby ap s entrada do
    primeiro jogador. */
public static final int LOBBY_TIMEOUT_MS = 20_000;

```

## 3.4 Interface e Lógica do Cliente

### 3.4.1 Lógica de execução e leitura de mensagens

Para maior eficiência, optamos por fazer uma thread de leitura e uma de escrita, isso faz com que possamos fazer as duas ações ao mesmo tempo, escrita e leitura.

```

public void run() {
    try {
        connectToServer();
        System.out.print("  Digite o seu nome: ");
        String name = scanner.nextLine().trim();
        this.pendingName = name;

        Thread threadLeitura = new Thread(this::
            readMessages);
        threadLeitura.setDaemon(true);
        threadLeitura.start();

        inputLoop();
    } catch (IOException e) {
        System.err.println("[Cliente] Erro fatal: " + e
            .getMessage());
    } finally {
        close();
    }
}

private void readMessages() {
    try {
        String line;
        while (running && (line = in.readLine()) !=
            null) {
            processServerMessage(line.trim());
        }
    } catch (IOException e) {
        if (running) {
            System.out.println("[Cliente] Conex o com
                servidor perdida: " + e.getMessage());
        }
    }
}
}

```

### 3.4.2 Inicialização da interface gráfica

Na GUI usamos Swing para uma interface visual, com painéis para força, botões de letras, entre outras coisas. A GUI usa Swing para uma interface

visual, com painéis para forca, botões de letras, etc.

```
public class ClienteForcaGUI extends JFrame {
    // Constantes de tema e estados
    private static final Color BG_DARK = new Color(15,
        15, 28);
    private static final Color CYAN = new Color(0, 210,
        215);
    // ...
    private int currentState = ST_MENU;
    private JPanel cardPanel;
    private HangmanPanel hangmanPanel;
    private FacePanel facePanel;
    private JLabel wordLabel;
    private JButton[] letterButtons;
    // ...
    private volatile boolean connected = false;
    private volatile String currentMask = "";
    private volatile int attemptsLeft = 6;
    ...
}
```

## Capítulo 4

# Demonstração do Jogo

### 4.0.1 Início do Jogo

Esta é a tela inicial do jogo:

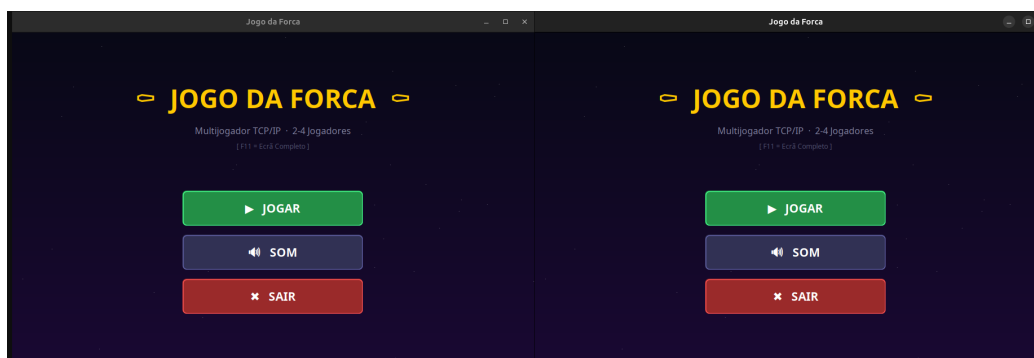


Figura 4.1: Tela de início

### 4.0.2 Aguardar Jogadores

Esta tela aparece quando estamos á espera dos outros jogadores:

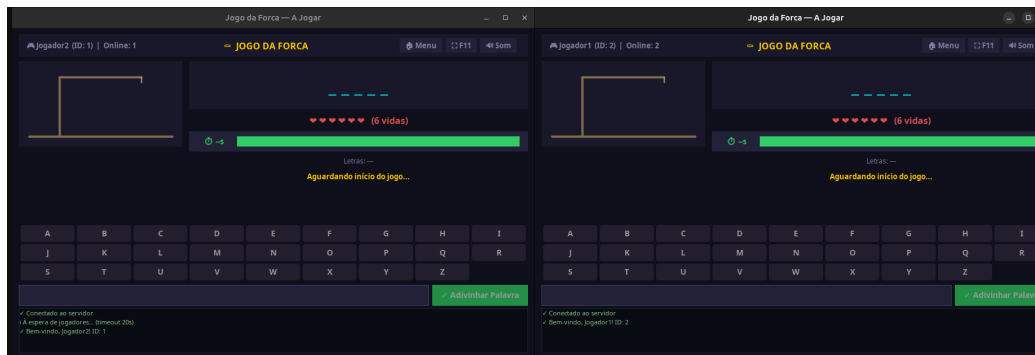


Figura 4.2: Tela de aguardar jogadores

### 4.0.3 Jogo em andamento

Estas duas telas é quando o jogo está em andamento em momentos de jogo diferentes:

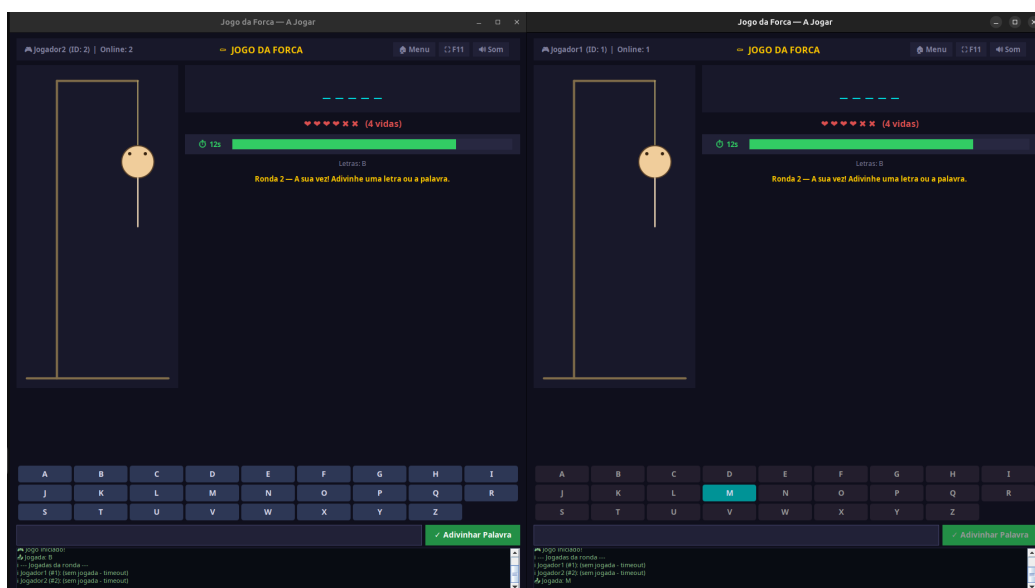


Figura 4.3: Tela de Jogo

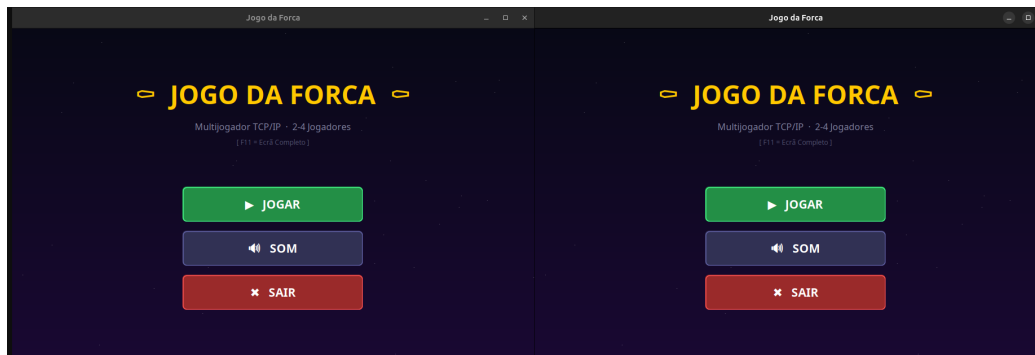


Figura 4.4: Tela de Jogo

#### 4.0.4 Cenário de vitória

Nesta tela, os jogadores venceram:

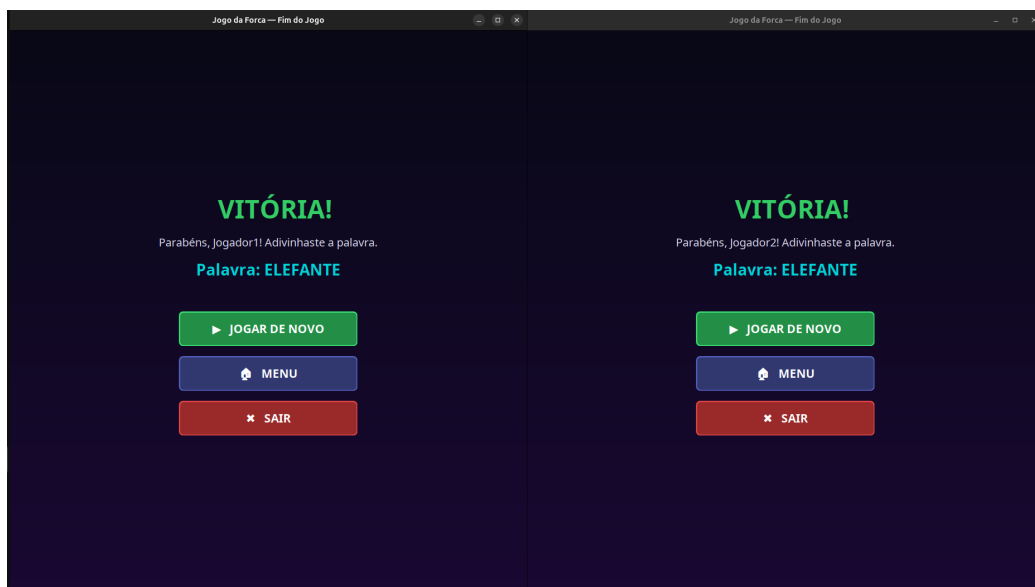


Figura 4.5: Tela de Vitória

#### 4.0.5 Cenário de derrota

Nesta tela, os jogadores perderam:

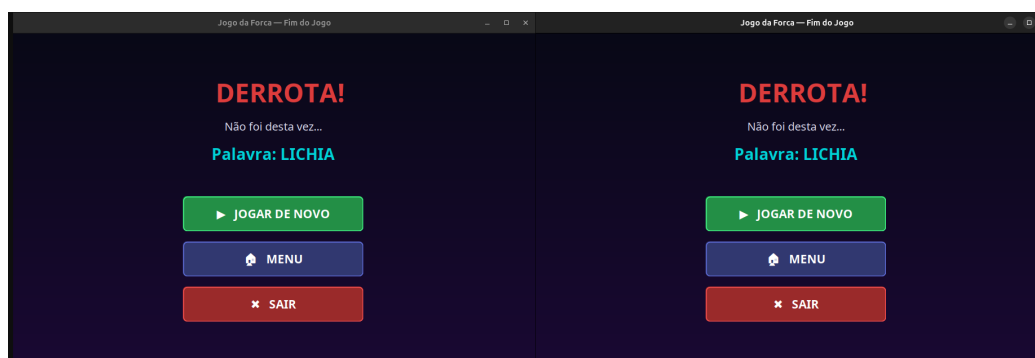


Figura 4.6: Tela de Derrota



# Capítulo 5

## Conclusão

### 5.1 Dificuldades Encontradas

A nossa maior dificuldade foi juntar todos os conhecimentos de sistemas distribuídos, visto que são um pouco mais complexas, mas o resultado final fez valer a pena, pelo controlo de race conditions, threads para maior eficiência e usabilidade.

### 5.2 Considerações Finais

Com isto tudo, nós concluimos que foi um projeto incrível de ser feito, onde podemos colocar em prática tudo o que fizemos até agora. Então, estamos orgulhosos do trabalho que fizemos.